Interoperable Cyber-Physical Multi-Agent Systems through Web of Things

Roman Binkert[1][0009−0000−5417−1225], Fady Salama[1][0000−0001−9225−6625], Ege Korkan[2][0000−0003−4910−4962], Sebastian Käbisch[2][0000−0002−0544−4204], and Sebastian Steinhorst[1][0000−0002−4096−2584]

[1] TUM School of Computation, Information and Technology, Technical University of Munich, Munich, Germany first.last@tum.de
[2] Siemens AG, Garching, Germany first.last@siemens.com

Abstract. With the rise of Internet of Things (IoT) systems, new paradigms are introduced to overcome their increasing complexity. One such paradigm is the Multi-Agent System (MAS), which consists of highly autonomous, reactive, and self-organizing agents. Many MAS frameworks exist but suffer from fragmentation and limit agents' interoperability, which remains a highly-discussed problem. Additionally, integrating Cyber-Physical Systems (CPSs) into MASs remains complex, limiting their physical applicability.

In this paper, we present Web of Multi-Agent Things (WoMAT), a novel concept to directly include the W3C Web of Things (WoT) standard into agent-to-agent communication. Our method leverages the standardized interoperability of WoT to seamlessly integrate heterogeneous MASs with each other and into CPSs while preserving their autonomy, decentralized coordination, and the expressive reasoning capabilities provided by AgentSpeak programming. We propose a systematic mapping between WoT Interaction Affordances (IAs) and AgentSpeak performatives, enabling agents to interact directly with diverse CPSs and exchange knowledge with other agents independent of the underlying communication protocols. Furthermore, our approach enables semantic interoperability between heterogeneous agents and CPSs by utilizing the descriptive powers of ontology-enriched WoT Thing Descriptions.

We provide an open-source implementation of our methodology, integrated with the JaCoMo framework for agent programming. Using multiple scenarios, we demonstrate and evaluate our approach and show that our method leverages MASs to be a more viable solution on the cyber-physical edge.

Keywords: Interoperability · Web of Things · Multi-Agent Systems · Cyber-Physical-Systems · Internet of Things

1 Introduction

The Internet of Things (IoT) is a significant driver for Industry 4.0. Via IoT, Cyber-Physical Systems (CPSs) are interconnected with each other, which leads

to higher automation and intelligence and, therefore, maximizes the efficiency in manufacturing and production. However, its exponential growth has led to multiple IoT platforms, manufacturers, and protocols, which has caused fragmentation. To tackle this interoperability challenge, the W3C proposes the Web of Things (WoT), which provides a standardized description of IoT device interactions. This allows us to interact with many different devices. However, a central organizer must orchestrate these different devices to work towards a common goal.

A well-studied approach for more autonomous and self-organizing systems are Multi-Agent Systems (MASs) with the Beliefs-Desires-Intentions (BDI) architecture. In MAS, multiple autonomous agents work together to find the best solution. The field of MASs is a long-studied field, and currently, multiple different agent development frameworks follow similar guidelines. However, the concrete implementation often varies, and especially when agents communicate over the web, a uniform interaction interface between the agents is not given, which hinders the collaboration of heterogeneous agents. To solve this issue, a uniform interaction interface between agents has been of interest to many research approaches [8]. Additionally, we have found that MASs often lack integration with the physical world since the integration and control of real devices often remain complex, which limits the applicability of MASs in CPS environments.

1.1 Problem Statement

The combination of WoT, for interoperability, and MASs, for autonomy and self-organization, has become of more interest in recent years, and it was shown that the combination can benefit both sides [11]. Recent approaches integrate the WoT into MAS by allowing agents to discover and communicate with new entities using the WoT Resource Description Framework (RDF) graphs [7], [15], [4]. However, these approaches require the agent programmer to directly include the WoT programming into the agent code and, therefore, require advanced knowledge of the WoT standards. Additionally, while allowing agents to communicate with Things, these approaches do not increase the interoperability between heterogeneous agents. Therefore, interoperability in heterogeneous agent environments is still a challenge.

1.2 Approach and Contributions

In this paper, we present our framework and its open-source implementation, Web of Multi-Agent Things (WoMAT), which utilizes the powers of WoT for diverse, protocol-independent multi-agent communication applications. As shown in Figure 1, our approach allows agents to interact with diverse IoT devices in a uniform, agent communication-like style. Furthermore, this communication approach can be used not only for agent-device communication but also for agent-to-agent communication. This increases the interoperability of heterogeneous agents by leveraging the descriptive and interoperability powers of WoT for MAS communication. Lastly, we show how the descriptive powers of WoT

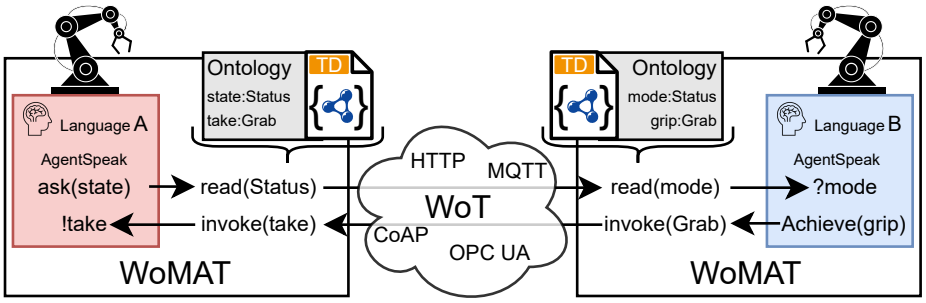

Fig. 1: The figure shows two agents (A_A, A_B), each running an instance of our framework, WoMAT. A_A and A_B use different internal namings for the same concept (e.g. `state`, `mode` for `Status`). Based on the ontology-enhanced TD of A_A, our framework in A_A maps the `ask(state)` request to the WoT IA `readProperty` representing `Status`, and finds and invokes the according property in A_B's TD. Our framework in A_B then translates this request to the naming of A_B (`mode`) and creates a source-annotated test goal in the belief base of A_B, which can then take action to respond.

and suiting ontologies can be used not only for communication and description of the agent but also to exchange knowledge and plans between heterogeneous agents. In particular, we perform the following contributions:

- We propose a mapping schema between AgentSpeak performatives and WoT Interaction Affordances (IAs) for protocol agnostic agent-device and agent-agent communication in Section 4.2.
- We show how our approach leverages the descriptive powers of Thing Descriptions (TDs) to semantically abstract from different internal namings of heterogeneous agents to automatically exchange knowledge and especially plans in Section 4.3.
- We evaluate our approach and open-source implementation with example use cases in Section 5.

The rest of the paper is structured as follows: We provide a short overview of the WoT features needed for this implementation, especially IAs and the TD, with its ability to integrate semantic tags in Section 2.1. An overview of the MAS concepts, especially for the JaCoMo agent development framework, is given in Section 2.2. We discuss related work in Section 3, present our approach in Section 4, evaluate in Section 5, and finally conclude in Section 6.

2 Background

Our framework is built upon two system designs, the WoT and MASs. This section will introduce both concepts.

2.1 WoT and TD

The WoT [13] is an amalgamation of multiple web standards aimed at providing a standardized interface in the heterogeneous landscape of IoT. At the core of WoT is the TD [14], a document that describes the interface of the underlying Thing. It is a JSON for Linked Data (JSON-LD) document and, therefore, is highly human- and machine-readable.

The three kinds of IAs define three ways of communicating or interacting with a Thing [13]. These IAs are defined as follows:

- **Property Affordance**: A property exposes the state or settings of a Thing. This state can be read and/or written. Additionally, it can be observable, informing observers of a change in this state. It can be used for, e.g., the value of a sensor (then read-only), the current status of a lamp (on, off), or to set the measurement interval of a sensor.
- **Action Affordance**: An action affects the state of the Thing or triggers a process on the Thing. This can be toggling a lamp or dimming a lamp over time.
- **Event Affordance**: The emission of an event leads to an asynchronous update to all subscribers. It is similar to an observable property but has no current state. It can be used, e.g., to inform that the Thing is overheating.

The semantic meaning of the data and IA can be described with so-called semantic annotations by using ontologies. Also, the structure of the input and output data of each IA can be precisely described with schemas and semantically annotated. Lastly, the protocol bindings define how the IA is mapped to the concrete protocol. Besides the IAs, a TD can hold meta information regarding the interaction with a Thing, like security definitions.

2.2 Multi-Agent Systems (MASs)

Agent programming generally focuses on creating autonomous, proactive, and reactive entities. In MASs, multiple agents work together and exchange information, and, therefore, it promises more flexible and scaleable solutions for complex tasks [10]. An established way of programming such agents is with the BDI architecture [6]. As shown in Figure 2, an agent perceives new beliefs about its environment through sensors or by communication from other agents. These beliefs about the environment are then compared to the agent's desires. If there is a difference between the current and desired states, the desire becomes an active goal. The agent will then seek a suitable plan to reach that goal and take action towards it.

Agent programming languages, like e.g. Jason, are commonly referred to as AgentSpeak and distinguish test (`?`) and achieve (`!`) goals. As shown in Figure 3, a plan has a plan trigger (`+!process(green)`) that defines when the plan is triggered, in this case, when a goal of name `!process` with input `green` is added. Once the plan is triggered, the actions of the plan (lines 2-5) will be executed in order. An agent programming language often comes with an agent programming

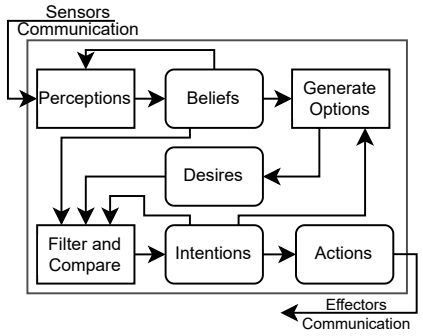

Fig. 2: Basic reasoning cycle of a BDI agent. Beliefs are added through perception or communication from other agents. The beliefs are then compared to the desires. If there is a difference, the agent will intend to change this, resulting in an action. Figure adapted from [5].

```
+!process(green) <-
    .print("Processing green");
    !moveTo(0.2, - 0.35, 0.3);
    !dropObject;
    !goHome.
```

Fig. 3: An agent plan programmed in Jason. If a new achievement goal to `process(green)` is added, the agent will follow this plan. It consists of four steps. First, it prints some information to the terminal, and then it adds three achievement goals, each needing to be fulfilled before moving to the next goal. With the first goal, it wants to move to some position, then drop the object, and finally go home.

framework, like JaCoMo, which provides further agent programming tools, such as a *mind inspector*, that allows the inspection of an agent's current beliefs (belief base) or intentions.

Agent Communication In MASs, multiple of the above-presented agents are present that can communicate with each other and exchange information. So-called Agent Communication Languages (ACLs) are used to define the information exchange between agents, An ACL unifies the message format and adds an ontology for all agents to communicate and interpret received messages. [10].

One central aspect of an ACL are the so-called *performatives*. A performative describes what an agent wants to express or reach with its message [1]. The Foundation for Intelligent Physical Agents (FIPA) defines several performatives like *inform, request, agree, and subscribe* [2]. However, the actual performatives vary between different agent-development frameworks, which also hinders the interoperability between agents.

For our implementation, we use the agent development framework JaCoMo with the following performatives:

- **Tell**: Agent A wants to share a belief with Agent B.
- **Ask**: Agent A wants to know whether Agent B knows something.
- **Achieve**: Agent A wants Agent B to achieve a goal.
- **AskHow**: Agent A wants to know if Agent B has a plan for a specified event.

3 Related Work

Integrating agents and MASs with real devices to enable them for manufacturing, the main potential application of MASs, or other cyber-physical applications is a long-studied problem [8].

Some approaches strive to directly integrate agents into Embedded Systems by allowing agents to control them via a serial port communication [16] and show that connecting via an IoT network can be a viable solution for, e.g., the management of crisis events [12]. However, we see agent programming as best applicable on a higher level of abstraction, like the WoT provides, especially if the target system is already IoT-enabled.

While WoT has not yet been used for agent-to-agent communication as we propose, several approaches integrate WoT for agent-to-device communication and see WoT as the first-class abstraction for IoT devices in MASs [9]. It has been demonstrated that with WoT-enabled MASs, agents can manage manufacturing systems, and based on the RDF representation of the TD, agents can interact with IoT devices and even refactor manufacturing systems on the fly. [9]. By Hypermedia, which can be described as RDF, relations among WoT devices can be shown. The framework *Yggdrasil* uses this to discover TD-described WoT devices and ways to interact with them at runtime. By the use of ontologies, knowledge about the devices can be represented in a standardized way and understood by the agents [8]. However, the representation of interacting with devices in AgentSpeak remains complex, limiting its descriptive powers. Hypermedia has been proven useful for agents, especially for planning with WoT devices. Different approaches provide artifacts that can discover things, how to interact with them, and comprehensively plan with them by using Knowledge Graphs [15] or through Linked Data navigation [7]. Therefore, the integration of WoT into MAS is an acknowledged approach that enables agents to interact with IoT devices in a standardized way. All these approaches focus on how agents can control devices and, mainly at runtime, find plans that include these devices. By introducing agent-to-agent communication through WoT, our contribution enables these works' discovery and planning mechanisms to be extended to heterogeneous agents in MASs.

The FIPA provides several standards for more interoperable MASs [17]. However, the standards focus on agent organization or message structure rather than the actual semantic interoperability of heterogeneous agents. Only a few approaches towards semantic interoperability of MASs have been made. But they introduce additional agents with the sole purpose of translating between different internal namings [3] instead of leveraging a standardized description of the agent's understanding.

4 Approach

In our approach, we combine MAS with WoT via a systematic mapping to easily connect heterogeneous agents and diverse CPSs. The capabilities of each agent

are uniformly described as IAs via the agent's TD, which allows to communicate with agnostic programmed agents via various protocols. On the other hand, CPSs and other heterogeneous agents described with a TD can be uniformly interacted with from agent plans in conventional AgentSpeak. In this section, we will elaborate on our fundamental mapping schema and how it leverages protocol-agnostic communication of WoT for uniform communication between agents and WoT-enabled devices. Additionally, we show how our approach utilizes semantic annotations in TDs to enable knowledge and plan exchange between semantically heterogeneous agents.

In WoT, a Thing that exposes its IAs through a TD is called exposing Thing, and a Thing interacting with that Thing is called consuming Thing. Following this naming convention, we use the terms *exposed agent* and *consuming agent*.

4.1 Framework Architecture

To enable MAS communication through WoT, our framework consists of two independent components that handle the mapping and semantic abstraction between internal AgentSpeak to WoT communication. The first component maps the AgentSpeak intent (performative) of the *consuming agent* to one of the WoT IA according to our mapping schema presented in Section 4.2 and shown in Figure 4. The second component semantically abstracts the concrete intent and data to a general ontology both agents understand, as presented in Section 4.3 and shown in Figure 5. Based on the abstracted intent and the annotated TD of the *exposed agent*, the *consuming agent* finds the according IA of the *exposed agent* and invokes it with the abstracted data. In the *consuming agent*, the same happens backward to translate everything from semantically abstracted, WoT IA to the internally used AgentSpeak of it. Notably, an agent can simultaneously act as an *exposed agent* in one information exchange while being the *consuming agent* in another.

4.2 Mapping Schema

To enable not only agent-to-device but also agent-to-agent communication through WoT while using the expressive AgentSpeak, we map AgentSpeak to WoT IAs in the *consuming agent* and also back from IAs to AgentSpeak in the *exposed agent*. Both parts will be described in the following.

Mapping from AgentSpeak to WoT IAs. As shown on the left side of Figure 4, most performatives can be mapped directly to IAs and are, therefore, universally applicable for all WoT devices. For some other performatives, we propose a dedicated action.

In AgentSpeak, the *ask* performative is used when an agent wants to request some information about a certain entity from another agent. In WoT, properties and events can be used to request information about something. Therefore, we map the *ask* performative to reading or observing a property or subscribing to an

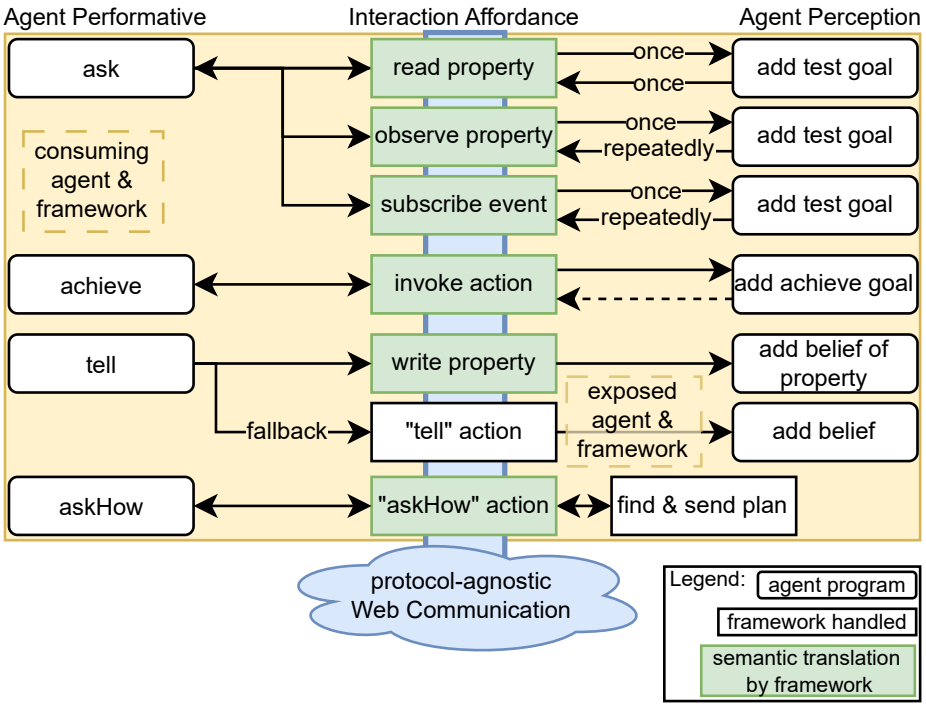

Fig. 4: The mapping schema from AgentSpeak to WoT IA and vice versa. The AgentSpeak in the *consuming agent* is first mapped to IAs, then sent through the WoT stack to the *exposed agent*, and there again mapped to AgentSpeak. Therefore, this approach also works if one side is not an agent. The framework handles semantic translation, particularly of plans and distribution of responses to the right *consuming agent* as described in Section 4.3. Most requests are forwarded to the *exposed agent* as translated, source-annotated beliefs or goals.

event, depending on how the information is exposed. If not indicated otherwise, the property is observed such that the agent gets updates repeatedly.

The *achieve* performative is used when the *consuming agent* wants to delegate a goal to the *exposed agent* and, therefore, asks the *exposed agent* to do something. In the WoT, the action IA triggers a process on the exposed Thing. Therefore, we map the *achieve* performative to invoking an action.

Both asking and delegating goals can or should lead to a response from the other agent or device. The response is represented in the agent's belief base as a source-annotated belief. In MASs, agents can also share information proactively. We map this to writing the according property of the *exposed agent*. If no semantically matching property is exposed, the information is communicated through a dedicated *tell* action. Finally, agents can exchange plans for achieving something, which is mapped to a dedicated *askHow* action. How a plan can be exchanged between semantically different agents will be elaborated in Section 4.3.

```
1 wotAskOne("ur10Thing", "position").
2 position(object(x(155),y(-205),z(80)))[source(ur10Thing)]
3 ?position(object(TargetPosition));
4 wotAchieve("agentB", "goTo", TargetPosition).
```

Listing 1: The agent asks for information (line 1) and the source-annotated response (line 2). The information can then be used as input to delegate a goal to another agent (lines 3 and 4). The commands work independently of the protocol and agent-to-agent or agent-to-device communication.

With the above-presented mapping, the desire of the *consuming agent* can be translated from AgentSpeak to invoking WoT IAs on other agents or conventional WoT devices. In the following, we will continue with the mapping from IAs back to AgentSpeak for the *exposed agent*.

Mapping from WoT IA to AgentSpeak. If an IA gets invoked on an agent's TD, this must be represented and forwarded to the agent so that it can react to this request. As shown on the right side of Figure 4, most IAs are mapped to source-annotated beliefs or goals, while our framework handles the other requests.

When a property is read, the consuming Thing expects some information, represented as adding a test goal to the agent's belief base. When the observation of a property is started, or when some Thing describes an event, this is not forwarded to the agent's belief base. Rather, the agent is expected to send updates by itself if something changes, and the framework keeps track of the information distribution. Similar to the mapping in the *consuming agent*, an invoked action adds a source-annotated achieve goal to the *exposed agent*. As shown before, a belief can be shared proactively by writing the according property or invoking the *tell* action with the belief as input. Both result in the addition of a source-annotated belief in the agent's belief base. If an IA has input data, this gets annotated as part of the belief or goal. Output data can be matched to the corresponding request via a generated message ID. Finally, the agent-specific *askHow* action triggers the plan exchange. This request, however, is not forwarded to the agent itself but handled by our framework. We will elaborate on this in detail in Section 4.3.

4.3 Semantic Abstraction

With the above-presented mapping schema, we can translate from AgentSpeak performatives to WoT IAs and vice versa. This allows agents to interact with other agents and WoT devices by using AgentSpeak. Based on this, we use the WoT semantic annotation functionality to enhance the interoperability between agents with heterogeneous naming conventions. In the following section, we show how this is done and how it can be used to translate heterogeneous plans.

We have N agents that use a different internal naming convention I_n for concepts C^{I_n}. By using the WoT as the communication layer as proposed above, agents are syntactically interoperable. With a mapping $M_{I_{n1}-I_{n2}}$ between the

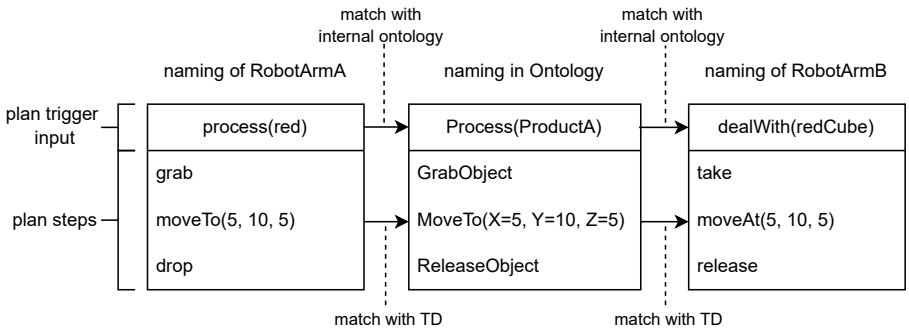

Fig. 5: The process of plan translation. Plan steps and inputs are mapped with the TD. Plan trigger and input are translated with a user-defined mapping.

different namings I_{n1} and I_{n2}, we can make these agents semantically interoperable. But for N agents, we need $\#M = \frac{N(N-1)}{2}$ mappings M.

As shown in Section 2.1, a TD can be enhanced with so-called semantic annotations, which again can be used to include an ontology in the TD. We use the semantic-annotated TDs to generate a mapping $M_{I_{n1TD}-O}$ between the internal naming I_{n1} and a shared ontology O. We reduce the required mappings to $\#M = N$ by this. Not all necessary concepts C might be represented in the TD of an agent, and, therefore, the generated mapping $M_{I_{n1TD}-O}$ might be incomplete. We allow a user-defined mapping $M_{I_{n1UD}-O}$ either in each function call or as a dictionary in the beginning for these concepts. For that, we have to map $M_{I_{n1}-O} = M_{I_{n1TD}-O} + M_{I_{n1UD}-O}$, which can map all concepts of the internal naming C^I_{n1} to the ontology.

If a *consuming agent* CA wants to get information for a concept C, it can send a request with the internal name of that concept $C^{I_{CA}}$. Our framework on the *consuming agent* side will then use the mapping $M_{I_{CA}-O}$ to translate the concept to the ontology C^O. Our framework will then crawl the TD of the *exposed agent* EA to find the according IA and invoke it. This will result in adding a source-annotated belief or goal of the concept in the internal naming of the *exposed agent* $C^{I_{EA}}$ to its belief base.

With these two mechanisms, the *exposed agent* and the *consuming agent* can work with their own internal naming conventions. Since the translation is done on the *consuming agent*'s side, it also works if the exposed Thing is a conventional WoT device. This works similarly for all performatives; only the translation of plans needs some extra steps, which we will elaborate on in the following.

Plan Translation. A plan consists of a plan trigger T_P, representing the higher level goal, with optional input J_P and one or several steps S_P, which all can have input J_SP. All of these can have a different internal name C^P. Therefore, contrary to beliefs and goals, plans are more complex to translate between different internal namings I.

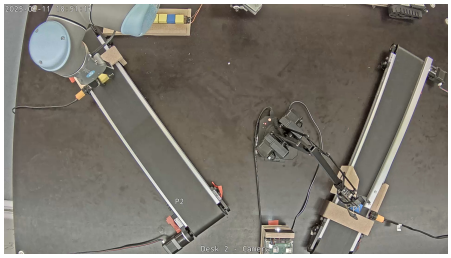

Fig. 6: The real manufacturing lane setup contains two robot arms (uarm and ur10) that should transport cubes between the two conveyor belts. All devices are agent-controlled.

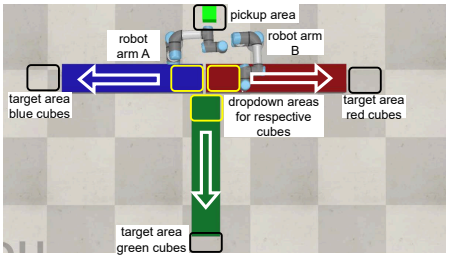

Fig. 7: The simulated robot arm setup contains three colored conveyor belts to which the two agent-controlled robot arms shall move the equally colored cubes.

In theory, the steps S_P could be anything. However, we state that only plans consisting of steps both agents can perform would make sense to translate. Therefore, we limit the translateable plans to only include goals exposed as IA in the TD or goals referring to third devices' IAs. Especially for steps S_P with inputs J_{SP}, the TD provides a structured way to annotate and find a mapping that would otherwise be hard to describe.

As shown in Figure 5, with the mapping of the TD M_{TD-O}, the plan steps can be translated similarly to beliefs and goals by our framework. Only for the plan trigger T_P and plan input J_P the user-defined mapping M_{UD-O} is necessary. Together with the above-presented translation for beliefs and goals, as well as the mapping schema from AgentSpeak to WoT IAs and back, we allow heterogeneous agents to communicate with each other as if they would have the same concrete naming.

5 Evaluation

In the previous section, we have shown how our approach enables agents to communicate over the WoT using conventional AgentSpeak, even if the agents use different internal namings. In this section, we evaluate our approach regarding interoperability between heterogeneous agents and WoT-enabled CPSs and performance.

We have implemented the approach with a CArtAgO artifact for the JaCoMo[3] agent-development framework for the agent side. For the WoT-side, we use the reference implementation of the WoT Scripting API node-wot[4].

[3] `https://jacamo-lang.github.io/` [4] `https://github.com/eclipse-thingweb/node-wot`

5.1 Experimental Setups

To demonstrate and evaluate the contributions of our approach, we conduct experiments in the following setups. The implementation of our approach, the code for the experiments, and videos showing the experiments are publicly available[5].

S_1: Real Cyber-Physcial MAS Setup: The first setup contains a real manufacturing setup with two robot arms and two conveyor belts, as shown in Figure 6. The task of the robot arms is to transport a cube from one conveyor belt to the other. A dedicated agent controls each device. While robot arms only consist of one Thing, the conveyor belts consist of a sensor for object detection and the conveyor belt with the motor. The agents communicate with the WoT representation of their corresponding device, and the robot arms communicate with the conveyor belt to know when a new cube is ready to pick up. The video demonstrating this use case can be found under the name *Real CPS Demo* in our above-linked GitHub repo.

S_2: Simulated Manufacturing Setup: The second setup contains two robot arms and three colored conveyor belts. The robot arms are controlled by agents $A_{R1,R2}$ with different internal namings $IR1, R2$. A WoT-enabled sensor can detect a new cube and its color in the pickup area and has to be contacted by the agents $A_{R1,R2}$. The robot arms should pick up the colored cubes from the pickup area and drop them on the dedicated conveyor belt, as shown in Figure 7. However, only A_{R1} has plans for blue and green cubes $P_{B,G}$. The example TD A_{R1} exposes its capabilities with I_{R1} as shown in Listing 2. The video demonstrating this use case can be found under the name *Simulated Manufacturing Demo* in our above-linked GitHub repo.

S_3: Protocol Agnostic Agents: This setup consists of four agents $A_{1,2,3,4}$. A_2 only supports HTTP, A_3 only supports CoAP, while A_1 supports both protocols. All of $A_{1,2,3}$ use different internal namings I. A_1 should communicate with both, A_2 and A_3, through our framework. A_4 is a standard JaCoMo agent with same naming as A_1 to which A_1 shall communicate for comparison.

5.2 Interoperability Evaluation

Using the above-presented setups, we evaluate our approach regarding protocol and semantic interoperability between heterogeneous agents and WoT devices.

Real CPS Integration: In S_1 and S_2, agents control their corresponding devices. In total, we controlled six different TD-described devices. All interactions with these devices, reading properties, invoking actions, and listening to events, could be done with a universal command for each interaction type, as shown in Listing 1.

Protocol Agnostic Agent Communication: In S_3, A_2 and A_3 use different protocols. However, A_1 can communicate with A_2 and A_3 using the same command, which does not include the protocol (similar to Listing 1). This proves that agents can communicate equally with agents and devices independent of the protocol.

[5] `https://github.com/tum-esi/WoMAT`

```
1  {"@type": "thing",
2  "title": "robot agent",
3  "@context": [{"eO": "exampleOntology"}],
4  "properties": { "state": { "@type": "eO:State" }},
5  "actions": {
6    "goHome": { "@type": "eO:GoHome"},
7    "moveTo": { "@type": "eO:MoveToPosition",
8      "input": { "properties": {
9        "x": { "@type": "eO:PositionX"},
10       "y": {  "@type": "eO:PositionY"},
11       "z": {  "@type": "eO:PositionZ"} }}},
12   "gripObject": { "@type": "eO:GrabObject" },
13   "dropObject": { "@type": "eO:ReleaseObject"},
14   "askHow": {},
15   "tell": {} }}
```

Listing 2: An example of an agent's TD. It contains our proposed agent-specific IAs `"askHow"` (Line 15) and `"tell"` (Line 16). The non-agent-specific IAs are annotated with an ontology. The rest of the TD is omitted for brevity.

Semantic Agnostic Agent Communication: In S_2, A_{R2} does not know how to process red and green cubes. Therefore, A_{R2} asks A_{R1} for the according plans. In the experiment A_{R1} and A_{R2} use different I. However, the plans in A_{R1} can be translated from I_{R1} to $IR2$ by our framework and then executed by A_{R2}. This shows that with our framework, knowledge and plans can be exchanged and used between heterogeneous agents.

5.3 Performance Evaluation

We evaluate the influence of our approach on the performance of the agent communication regarding the processing time and the necessary lines of code.

Processing Time Evaluation: To evaluate the processing time, we measure the translating times for plans with different amounts of steps and compare it to the total execution time, including the search in the agents plan library and the communication between AgentThing and CArtAgO artifact. As shown in Figure 9, the plan translation time grows linear with the number of steps but is marginal compared to the total execution time. The processing time performance evaluation was done using a laptop with an 11th Gen Intel Core i7-11800H processor containing 8 cores and running at 2.30 GHz frequency, 16 GB of DDR4 RAM, and NVIDIA GeForce RTX 3060 Laptop GPU.

Code Line Evaluation: Our approach provides protocol and semantic agnostic communication between agents. When the agents use the same internal naming, they can still communicate protocol agnostic through the web with our framework. As shown in Figure 8, a similar AgentSpeak is used. Only the TD address and the agent's own TD must be defined once. When the agents use different internal naming, the same is still needed, but the TD needs to be enhanced with ontology tags and mappings that are not defined there need to be defined additionally. However, in the agent programming, the necessary code stays the same. This shows that for homogeneous agents, our approach provides

```
// Jason
.send(agentB, askOne, status).   // A asks
// answer by agent B
+?status[source(Source)]: status(Status) <-
    .send(Source, tell, status(Status)).
// WoMAT
wotAskOne(agentB, status).  // A asks
// answer by agent B
+?status(MsgID): status(Current) <-
    wotSendTestResponse(Current, MsgID).

// define TD address for agentB
{"agentB": "http://localhost:8080/agentB"}
```

Fig. 8: The code lines needed for agent communication in plain Jason and with our framework. The *consuming agent* only needs the TD address of `agentB` additonally. The *exposed agent* needs a TD as shown in listing 2. Therefore, the AgentSpeak overhead is minimal.

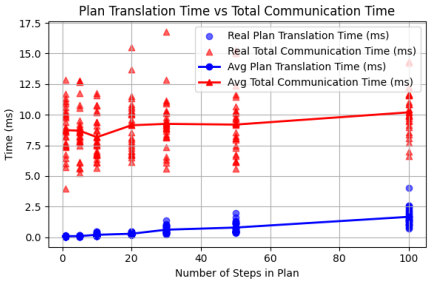

Fig. 9: The time needed for just the plan translation grows linear with the number of steps the plan has. It is marginal compared to the total communication time, which shows that the overhead for the semantic translation is small.

a protocol-agnostic web communication with minimal overhead, and especially for heterogeneous agent programming, the added overhead stays minimal.

6 Conclusion

This paper presents WoMAT, our novel approach that enables heterogeneous agent-to-agent communication through WoT. Our method uniquely enables agents to interact with WoT devices with the expressive powers of AgentSpeak.

We demonstrate how our approach maps WoT IAs to AgentSpeak performatives, leveraging the descriptive powers of the TD to provide uniform and ontology-enriched agent descriptions. This enables semantically heterogeneous agents to exchange knowledge and plans while keeping their internal naming conventions. Additionally, the mapping enables agents to utilize AgentSpeak's expressive reasoning to interact with other agents and conventional WoT-enabled devices, independent of the underlying protocols and semantics.

We provide an open-source implementation of our approach with the JaCoMo agent framework and the node-wot scripting API. Our evaluation across multiple scenarios demonstrates that WoMAT effectively integrates CPSs into MASs and enables the exchange of knowledge and plans between heterogeneous agents. In the future, our approach could be combined with other frameworks, such as [7], to extend their agent-to-device planning and discovery capabilities to agent-to-agent interactions.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. FIPA ACL Message Structure Specification. http://www.fipa.org/specs/fipa00061/SC00061G.html#_Toc26669706 (nd), accessed: 2024-04-02
2. FIPA Communicative Act Library Specification. http://www.fipa.org/specs/fipa00037/PC00037E.html#_Toc505673625 (nd), accessed: 2024-04-02
3. Amrani, N.E.A., Snineh, S.M., Youssfi, M., Abra, O.E.K., Bouattane, O.: Interoperability model between heterogeneous MAS platforms based on mobile agent and reinforcement learning. In: ICDS (2021). https://doi.org/10.1109/ICDS53782.2021.9626723
4. Bienz, S., Ciortea, A., Mayer, S., Gandon, F., Corby, O.: Escaping the streetlight effect: Semantic hypermedia search enhances autonomous behavior in the web of things. IoT '19 (2019). https://doi.org/10.1145/3365871.3365901
5. Boissier, O., Bordini, R.H., Hübner, J.F., Ricci, A.: Multi-agent oriented programming - the jacamo platform. https://www.emse.fr/~boissier/enseignement/maop16/pdf/aop.pdf (2016)
6. Bordini, R.H., Hübner, J.F.: BDI agent programming in agentspeak using jason (2006). https://doi.org/10.1007/11750734_9
7. Charpenay, V., Zimmermann, A., Lefrançois, M., Boissier, O.: Hypermedea: A framework for web (of things) agents. WWW '22 (2022). https://doi.org/10.1145/3487553.3524243
8. Ciortea, A., Boissier, O., Ricci, A.: Engineering world-wide multi-agent systems with hypermedia. In: EMAS (2019). https://doi.org/10.1007/978-3-030-25693-7_15
9. Ciortea, A., Mayer, S., Michahelles, F.: Repurposing manufacturing lines on the fly with multi-agent systems for the web of things. EMAS (2018)
10. Dorri, A., Kanhere, S., Jurdak, R.: Multi-agent systems: A survey. IEEE Access **6** (2018). https://doi.org/10.1109/ACCESS.2018.2831228
11. Kampik, T., Mansour, A., Boissier, O., Kirrane, S., Padget, J., Payne, T.R., Singh, M.P., Tamma, V., Zimmermann, A.: Governance of autonomous agents on the web: Challenges and opportunities. ACM Transactions on Internet Technology (2022). https://doi.org/10.1145/3507910
12. Lazarin, N.M., Alexandre, T., Moreira de Paiva, M., Pantoja, C.E., Viterbo, J., Bernardini, F.: A decentralized agent-based model for crisis events using embedded systems. In: PAAMS (2025). https://doi.org/10.1007/978-3-031-70415-4_14
13. Matsukura, R., McCool, M., Toumura, K., Lagally, M.: Wot architecture 1.1. Recommendation, W3C (2023), https://www.w3.org/TR/2023/REC-wot-architecture11-20231205/
14. McCool, M., Korkan, E., Käbisch, S.: Wot thing description 1.1. Recommendation, W3C (2023), https://www.w3.org/TR/2023/REC-wot-thing-description11-20231205/
15. Noura, M., Siegert, V., Gaedke, M.: Wat: Autonomous hypermedia-driven web agents for web of things devices. In: CEUR Workshop Proceedings (2021), https://ceur-ws.org/Vol-3111/short6.pdf
16. Pantoja, C.E., Souza de Jesus, V., Lazarin, N.M., Viterbo, J.: A spin-off version of jason for iot and embedded multi-agent systems. In: Intelligent Systems (2023). https://doi.org/10.1007/978-3-031-45368-7_25
17. Poslad, S., Charlton, P.: Standardizing agent interoperability: The fipa approach (2001). https://doi.org/10.1007/3-540-47745-4_5